This chapter allowed you to review best practices to follow when you are developing Dapps:

- Carefully examine the problem, evaluate the context, analyze the traditional solutions, and discuss any alternatives, if available.
- Design before you start developing the solution. Use standard diagrams such as a contract diagram and state diagram to represent the design.
- Use the FSM diagram and contract diagram as guidelines for developing the smart contract and user interface.
- Use standard directory structure and locations for the critical files. For XYZ-Dapp, use XYZ-contract, XYZ-app, XYZ.sol, and app.js.
- Use modifiers for representing rules. Modifiers revert transactions that don’t meet the rules, so they help prevent the unnecessary recording of transactions on the blockchain.
- Use and emit event definitions to indicate significant milestones. These events emitted are recorded on the block and can be used for UI notification as well as postdata analysis.
- Design the smart contract with only the required data structures and operations. The smart contract has to be concise and precise, with simple, straightforward logic. Avoid loops and complex computations by creatively moving these operations to the nonblockchain components of the Dapp, such as to app.js.
- Use the Truffle and Ganache test environments to test the smart contract’s operation before deploying the Dapp on a public network such as Ropsten.